面向东盟跨境物流的突发气象预警与多路线协同调度平台 · 技术文档

> 参赛组别：智能应用类 用途：技术实现与复现说明。关键密钥已脱敏（以 `<...>` 占位），全部步骤可复现。
>
> 与提交要求的对应关系：
>
> | 提交要求 | 对应章节 |
> | --- | --- |
> | 环境配置说明 | 二（环境要求）、四（配置项与脱敏模板）、十（依赖获取与网络/代理矩阵） |
> | 依赖列表 | 三（后端 / 前端 / 本地模型依赖）、10.1（获取渠道与镜像） |
> | 部署步骤 | 五（后端、大屏、司机端 Web、Android、离线演示）、11.1（从零复现） |
> | API 接口清单 | 六（19 个控制器 / 96 个端点，由 `tools/_api_inventory.mjs` 扫描生成） |
> | 关键配置可脱敏且可复现 | 10.3（脱敏对照与密钥边界）、11.1（无凭证跑通全流程）、11.2（环境差异与已知坑） |

一、技术栈总览

层次	技术选型	版本
后端核心	Spring Boot	4.1.0
语言/运行时	Java	21（Gradle toolchain 指定）
图计算	自建邻接表 + 优先队列 A*（可采纳启发式：直线距离 ÷ 全图最高限速，最优解与纯 Dijkstra 完全一致，只是剪掉无向扩散）；baseline/current 双路多核并行；GraphHopper（OSM PBF 坐标算路，默认关闭）	GraphHopper 10.2
AI 智能体	Ollama（本地私有化）+ Qwen2.5-14B；三智能体 `RiskAgent → SolutionAgent → TouchAgent` 串行，由 `DecisionOrchestrator` 编排并经 SSE `agent-status` 逐条点亮	qwen2.5:14b / deepseek-chat
PC 调度大屏	Vue 3 + Vite + Leaflet（Canvas 渲染保护 + 底图四级降级链）	Vue 3.5 / Vite 8 / Leaflet 1.9.4
实时通道	Spring MVC `SseEmitter` 单条流（`/api/agent/events`）承载 6 类业务事件：`route-update` / `task-assigned` / `agent-status` / `outreach-update` / `canal-block` / `canal-recover`	—
移动端司机 App	Vue 3 + Capacitor（当前落地 Android）	Capacitor 8.5
数据层	JSON 数据 + 内存/磁盘瓦片缓存 + 决策预生成缓存	—
构建	Gradle（后端）、npm/Vite（前端）	—

二、环境要求

- **JDK 21+**（项目 toolchain 锁定 Java 21；更高版本 JDK 亦可编译运行）
- **Node.js 20+**（前端 Vite 8 构建；实测 Node 26 可用）
- **Android SDK**（仅打包司机端 APK 时需要）
- **Ollama**（本地大模型推理，离线演示必需）

三、依赖清单

3.1 后端（`../build.gradle`）

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.0'
    id 'io.spring.dependency-management' version '1.1.7'
}
java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }

dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-web'
    developmentOnly  'org.springframework.boot:spring-boot-devtools'
    compileOnly      'org.projectlombok:lombok'
    annotationProcessor 'org.projectlombok:lombok'

    // 图算法（依赖已声明：当前路径规划以自建 Dijkstra 为主）
    implementation 'org.jgrapht:jgrapht-core:1.5.2'
    implementation 'org.jgrapht:jgrapht-io:1.5.2'

    // GraphHopper：直接读取 OSM PBF 做坐标算路
    implementation('com.graphhopper:graphhopper-core:10.2') {
        exclude group: 'org.slf4j', module: 'slf4j-simple'
    }
    // OSM PBF 解析
    implementation 'org.openstreetmap.osmosis:osmosis-core:0.49.1'
    implementation 'org.openstreetmap.osmosis:osmosis-pbf:0.49.1'
    implementation 'org.openstreetmap.pbf:osmpbf:1.5.0'

    implementation 'com.google.code.gson:gson:2.10.1'   // jackson 由 Spring Boot BOM 管理

    testImplementation 'org.springframework.boot:spring-boot-starter-test'
}

tasks.named('bootRun') { jvmArgs = ['-Xms2g', '-Xmx6g', '-XX:+UseZGC'] }
```

3.2 前端（`../frontend/package.json`）

```
{
  "dependencies": {
    "@capacitor-community/text-to-speech": "^8.0.2",
    "@capacitor/geolocation": "^8.2.2",
    "axios": "^1.19.0",
    "leaflet": "^1.9.4",
    "vue": "^3.5.40"
  },
  "devDependencies": {
    "@capacitor/android": "^8.5.2",
    "@capacitor/assets": "^3.0.5",
    "@capacitor/cli": "^8.5.2",
    "@capacitor/core": "^8.5.2",
    "@capacitor/ios": "^8.5.2",
    "@vitejs/plugin-vue": "^6.0.8",
    "vite": "^8.2.0"
  }
}
```

> **底图不使用 MapLibre / 矢量瓦片 GPU 渲染。**四级降级链全部是 Leaflet 标准 `L.tileLayer`，源定义集中在 `../frontend/src/utils/tileSources.js`（大屏与司机端共用同一份顺序）： 底图1 天地图影像 `img_w` +注记 `cia_w` → 底图2 天地图矢量 `vec_w` +注记 `cva_w` → 底图3 OSM 在线栅格 → 底图4 离线瓦片（ `/tiles/{z}/{x}/{y}.jpg`，EPSG:4326，由 `WebConfig` 静态托管）。 天地图三种一律走同源后端代理 `/api/tianditu/{layer}/{z}/{x}/{y}.png`，**浏览器包内零密钥。**

3.3 本地大模型

```
# 安装 Ollama 后拉取模型（约 9GB）
ollama pull qwen2.5:14b
# 服务默认监听 http://localhost:11434
```

四、配置说明（`../src/main/resources/application.properties` + `.env.local`）

后端通过 `spring.config.import=optional:file:./.env.local[.properties]` 读取根目录 `.env.local` 环境变量。以下为**脱敏**后的关键项:

配置项	说明	默认值 / 示例（已脱敏）
`server.address`	监听地址，`0.0.0.0` 使手机同 WiFi 可连	`0.0.0.0`
`weather.real.base-url`	大赛官方 CRA40 实况/预报接口（GOWFS 共用）	`http://<官方地址>:8001`
`weather.real.token`	官方气象接口 Token（**仅存后端**）	`<气象TOKEN>`
`weather.real.observation-time`	实况起报时次 `YYYYMMDDHHMM`	`202412312000`
`weather.source`	数据源切换：`contest-observation` / `open-meteo` / `simulated`	`contest-observation`
`weather.real.ttl-minutes`	实况内存缓存时长	`10`
`weather.real.min-request-interval-seconds`	**每源各自**最小回源间隔（切源不被上一个源的窗口挡住）	`60`
`weather.real.min-source-switch-gap-seconds`	跨源最小回源间隔（防“来回切源”把窗口变摆设）	`5`
`weather.real.circuit-failure-threshold` / `circuit-open-seconds`	连败开熔断阈值 / 静默时长（**熔断态也按源各自算**，官方 Token 失效不连带封 Open-Meteo）	`3` / `300`
`weather.forecast.min-request-interval-seconds`	GOWFS 3h 预报独立节流（与实况不抢额度）	`1800`（静默 `600`）
`weather.forecast.run-time` / `hours`	预报起报时次（仅 `08:00` / `20:00`）与累计时效（3 的倍数，最多 5 个）	`202412312000` / `3,6`（即未来 6h）
`weather.watchdog.enabled` / `interval`	后台气象守护（**不依赖客户端轮询**的真实检测触发源）	`true` / `60s`（启动延迟 20s）
`ollama.enabled` / `model` / `url`	本地大模型开关与 OpenAI 兼容端点（离线演示置 true）	`true` / `qwen2.5:14b` / `http://localhost:11434/v1/chat/completions`
`deepseek.fallback.key`	在线 DeepSeek 兜底 Key（**未填即视为该通道未启用**，直接降级不假装成功）	`<DEEPSEEK_KEY>`
`decision.cache.dir`	多智能体预生成缓存目录	`decision-cache`
`tianditu.tk` / `tianditu.keys`	天地图单密钥 / **密钥池**（轮转 + per-key 熔断，仅存后端）	`<TK1,TK2,...>`
`tianditu.cache-dir` / `mem-cache-entries` / `referer` / `timeout-seconds`	瓦片磁盘缓存目录 / 内存 LRU 上限 / 密钥未授权域名时的兜底 Referer / 单张超时（超时即冷却该 key 并换 key 重试，全池失败才 503 让前端降级）	`../data/tile-cache` / `20000` / `<已授权域名>` / `6`
`geocode.nominatim.enabled` / `endpoint` / `proxy` / `region-bias`	越南街道/店名兜底 Nominatim（天地图对越南 POI 几乎无结果）；代理端口留空则直连；裸地名命中外地同名点时用城市前缀纠偏	`true` / 官方端点 / `<可空>` / `南宁市`
`demo.guard.enabled` / `deny-osm-loader` / `strict-limit-per-minute` / `normal-limit-per-minute`	公网演示护栏：总开关 / 封死 311 MB PBF 加载入口 / 改全局状态的写接口限流 / 其余写接口限流	`true` / `true` / `20` / `180`
`waterway.source` / `official.api-base` / `official.token` / `poll-interval-ms`	平陆运河通航观测接入：`mock`（手动推送验证同一链路）/ `official`；轮询与水文站 15 min 上报同频（**mock 模式不轮询**，免得定时把演示剧本打回正常）	`mock` / `<待协议获取>` / `<预留>` / `900000`
`osm.loader.enabled`	GraphHopper OSM PBF 加载开关（坐标算路模式）	`false`

> **合规说明**：气象 Token、天地图密钥池、DeepSeek Key、预留的水运官方 Token 全部只存后端环境变量（真值写本机 `.env.local`，已 `../.gitignore`），前端与 APK 构建产物零密钥；未配置的通道一律按“未启用”处理并自动降级，不使用占位凭证伪称调用成功。 AI 降级链为 **本地 Ollama（优先，数据不出境）→ 在线 DeepSeek → 历史 AI 文案池轮换 → 规则模板** 四级；实际命中哪一级就在界面标注哪一级，三智能体的汇总标签取其中远程程度最高的一级，不存在“本地推理”标签下实际走了在线通道的情况。

`.env.local` 模板：仓库根目录已提供可直接复制的脱敏模板 `.env.example`（`copy .env.example .env.local`），其中每一项都带“留空会怎样”的注释。手工填写时的最小集合如下：

```
WEATHER_REAL_BASE_URL=http://<官方地址>:8001
WEATHER_REAL_TOKEN=<气象TOKEN>
WEATHER_REAL_OBSERVATION_TIME=202412312000
OLLAMA_ENABLED=true
OLLAMA_MODEL=qwen2.5:14b
TIANDITU_KEYS=<TK1,TK2,TK3,TK4,TK5>
DEEPSEEK_FALLBACK_KEY=<可选>
GEOCODE_NOMINATIM_PROXY=<本机代理端口，可留空>
# 可选：tianditu.mem-cache-entries=20000、WEATHER_FORECAST_RUN_TIME=202407152000（雨季真实降水样例）
```

前端**不需要任何环境变量与密钥**：`../frontend` 下不存在 `.env.local`，全部前端代码只读取 `import.meta.env.PROD` 一个构建期常量；历史上遗留的 `VITE_TIANDITU_TK` 已随“浏览器直连天地图”方案一起移除（改走后端同源代理后无任何代码读取它）。

五、部署与运行步骤

> 前提：clone 后先准备 `.env.local`（可直接用根目录 `.env.example` 复制，全部留空也能启动）。 JDK 不需额外配置：`../gradle.properties` 已启用 toolchain 的 auto-detect / auto-download，**不写死任何开发机的 JDK 绝对路径**； 若本机已装多个 JDK 需指定，取消 `org.gradle.java.home` 一行的注释并改为本机路径即可（默认保持注释状态）。

5.1 后端服务

```
# Windows
gradlew.bat bootRun
# Linux / macOS
./gradlew bootRun
```

- 启动后监听 `http://<本机IP>:8080`，`GET /ping` 返回 `OK` 即成功。
- 启动时自动加载 `resources/data/vietnam-road-network.json` 构建拓扑图（`StartupWarmup` 预热）。

5.2 PC 调度大屏 / 司机端（Web）

```
cd frontend
npm install
npm run dev          # 开发：Vite 热更新，默认 http://localhost:5173
# 或构建产物（后端会从 file:frontend/dist 托管）：
npm run build        # 输出到 frontend/dist
```

- 大屏入口：`http://<本机IP>:8080/`
- 司机端入口：`http://<本机IP>:8080/driver.html`

> 注意：Spring 静态托管读的是**已构建的 dist**，改前端代码后需 `npm run build` 重新构建；开发调试则用 Vite dev server。

5.3 司机端 Android APK（瘦客户端）

```
cd frontend
npm run build
npx cap sync android
cd android
gradlew assembleDebug --no-daemon
# 产物：android/app/build/outputs/apk/debug/app-debug.apk
```

- `capacitor.config.json` 的 `server.url` 指向**运行后端的电脑局域网 IP**（示例 `http://192.168.200.140:8080/driver.html`）。演示前需把该 IP 改为当前后端机器 IP，手机与电脑连同一 WiFi 即可。
- 瘦客户端机制：APK 运行时 WebView 加载电脑上的 dist，因此前端改动只需重新 `npm run build` + 后端在跑，手机重开 App 即更新，无需重装 APK。

5.4 离线演示（拔网线）清单

1. 提前 `ollama pull qwen2.5:14b` 并让 Ollama 随系统启动（`OLLAMA_ENABLED=true`）。
2. 提前拉取一次气象数据 / 或切 `weather.source=simulated`。
3. 提前生成决策缓存：`../tools/pregen_agent_cache.ps1` 按 scenarioId 跑好三组场景，产出 `decision-cache/*.json`（`scenario_heavy_rain / scenario_light_rain / scenario_dual_risk`）与文案池 `recommendation-pool.json`。**在途行程不读缓存**（缓存里的推荐没考虑车的位置）。
4. 天地图瓦片：已两级缓存（内存 + `../data/tile-cache` 磁盘），演示前预热热门瓦片。
5. 断网后重复核心流程验证：本地推理、本地算路、本地瓦片全部可用；界面上的来源/通道徽标应如实变为「模拟降级 / 缓存 / 模板」而不是继续谎称实时。

六、API 接口清单

> 共 **19 个控制器 / 96 个端点**（由 `node tools/_api_inventory.mjs` 从 `controller/` 包直接扫描生成，与代码同步）。 计数含无路径的根映射（如 `@GetMapping` 不带括号参数的 `GET /api/geocode`、`GET /api/knowledge`）；取证脚本早期版本因正则强制要求（ 而漏计这类端点，已修正。 统一前缀 `/api`；实时事件走 SSE（`text/event-stream`）。

6.1 算路与路线状态 `RouteController`（`/api/routes`，同时兼容别名 `/api/route`）（12）

方法	路径	说明
POST	`/plan`	路径规划（两种入参二选一：`originId+destinationId` 节点模式，或 `startLat/startLng/endLat/endLng` 坐标模式——后者先吸附最近路网节点，超 5 km 报 400「该区域暂未覆盖路网」）；返回 baseline/current/extraHours、风险逐段、`routeGeoJSON`、吸附信息
GET	`/plan-with-weather`	带气象的方案（AI 已移出关键路径，算路不等模型）
GET	`/plan-real`	GraphHopper 坐标算路（`fromLat/fromLon/toLat/toLon`），需 `osm.loader.enabled=true` 且 PBF 已加载
GET	`/candidates`	多候选路线（推荐 / 备选 / 最快）
POST	`/plan-sample`	样例路径规划（演示固定起终点）
POST	`/detour`	绕行计算（禁用指定风险边的局部重算）
GET	`/status`	OSM 加载器状态（`enabled/loading/loaded + message`，对应顶栏「OSM 状态」）
POST	`/block`、`/unblock`	手动熔断 / 解除某条边（`penaltyMultiplier` 默认 100）
POST	`/osm/start`、`/osm/stop`	GraphHopper OSM PBF 坐标算路加载开关（**公网演示被护栏直接 403**，它会加载 311 MB PBF）
GET	`/network`	路网拓扑（节点/边，供骨骼图与调试）

6.2 运输任务与在途 `TaskController`（`/api/task`）（5）、`TripController`（`/api/trip`）（4）

方法	路径	说明
POST	`/api/task/dispatch`	公司派单 → SSE 广播 `task-assigned`（默认剧本：桂A·D12345 / 阿明 / 18 t 冷鲜 / 南宁→河内）
GET	`/api/task/current`	司机端拉取当前任务（SSE 断开时的兜底回到任务模式）
POST	`/api/task/accept`、`/api/task/reject`	司机接单 / 拒单（带原因），回执回流大屏与决策日志
POST	`/api/task/reset`	演示复位：撤销任务（界面「撤销派单」）
POST	`/api/trip/start`、`/api/trip/progress`	开始行程 / 上报在途进度（司机端约 30 s 一次；超 5 min 视为不在途）
GET	`/api/trip/signal`、POST `/api/trip/reset`	读当前行程信号 / 复位

6.3 实时事件与三智能体 `AgentController`（`/api/agent`）（4）、`AgentAnalysisController`（1）

方法	路径	说明
GET	`/api/agent/events`	**SSE** 唯一长连接；首帧 `init`，之后推 `route-update` / `task-assigned` / `agent-status` / `outreach-update` / `canal-block` · `canal-recover`
POST	`/api/agent/register`、`/api/agent/unregister`	关注路线注册 / 注销（大屏与司机端各自登记自己在看的路线）
GET	`/api/agent/status`	守护与重算运行状态（不含密钥）
GET	`/api/agents/analysis`	三智能体决策分析（风险研判 / 方案生成 / 触达）+ 通道标签

事件与两端的适配关系（逐事件比对后端 `SseEmitter.event().name(...)` 与两端 `addEventListener(...)` 得出）：

事件	后端广播位置	调度大屏 App.vue	司机端 DriverApp.vue
`init`	`RouteAgentService` 连接建立时	不监听（当前活动起终点由各端自行 `POST /api/agent/register` 确定，不依赖首帧）	不监听（同上，另有 `GET /api/task/current` 兜底）
`route-update`	`RouteAgentService` 风险变化重算后（300 ms 防抖）	✓ 刷路线与风险着色	✓ 刷导航与换线弹窗
`task-assigned`	`TaskController` 派单后	✓ 任务卡状态	✓ 进任务模式
`agent-status`	`DecisionOrchestrator` 逐智能体点亮	✓ 三智能体面板	— 司机端不展示推理过程，按设计不监听
`outreach-update`	`OutreachService` 分级叫应状态变化	✓ 触达进度的时间线	✓ 指令弹窗
`canal-block` / `canal-recover`	`DecisionSandboxService` 禁航判定与解除	✓ 水运层禁航态	✓ 司机端禁航提示与换路

> 结论：除 `init` 为预留帧外，六类业务事件在「后端广播 ↔ 两端监听」两侧名称与语义完全一致，无孤立事件、无未接事件。

6.4 气象与风险 `WeatherController`（`/api/weather`）（13）

GET `/source`（所选源与真正服务源分开回显）、POST `/source/{sourceId}`（切换源）、**GET `/rate-stats`**（节流审计四计数：真实回源/缓存命中/节流拦下/熔断打开）、GET `/real`、POST `/real/refresh`、POST `/risk`、GET `/risk`、DELETE `/risk/{edgeId}`、DELETE `/risk`、GET `/scenarios`、POST `/scenario/{scenarioId}`、DELETE `/scenario/{scenarioId}`、POST `/deepseek/scan`

6.5 官方气象代理 `ContestWeatherController`（`/api/v1`）（4）

GET `/variables`、`/observations`（11 要素，单次最多取 3 个）、`/forecasts/3h`（`run_time` 仅 08:00/20:00，`forecast_hours` 取 3 的倍数、最多 5 个且 ≤168）、`/rate-stats`。同源代理且要求携带与后端一致的 Bearer，第三方借不到额度；401 立即熔断。

6.6 决策沙盘 `SandboxController`（`/api/sandbox`）（15）

POST `/push`（灾害剧本注入）、`/push-scenario`、`/push-live`（官方接口推送）、`/sim/enter`、`/sim/exit`、`/sim/params`（七要素滑块）、`/typhoon`、`/typhoon/clear`、`/rainfall`（口岸断面降雨）、`/water`（运河通航条件）、`/water/block`、`/water/clear`、`/reset`；GET `/state`（全局态势快照）、`/forecast-status`（GOWFS 通道自检，**不含密钥**）

6.7 水运与联运 `CanalController`（`/api/canal`）（6）、`WaterwayController`（`/api/waterway`）（3）

GET `/canal/tributaries`（支流风险）、POST `/canal/tributaries/apply`、GET `/canal/tributaries/live`（支流实况）、POST `/canal/tributaries/apply-live`、GET `/canal/intermodal`（陆水联运方案）、GET `/canal/corridor`（走廊与全域受阻建议）； GET `/waterway/status`（当前数据源 mock/official）、POST `/waterway/push`（手动推送验证同一链路）、POST `/waterway/poll`（触发一次轮询）

6.8 分级叫应与留痕 `OutreachController`（`/api/outreach`）（6）

POST `/dispatch`（按方案下发五类角色）、POST `/confirm`（回执确认）、GET `/status`（逐角色 已送达/已确认/外呼中）、GET `/message`（中越双语指令与权益保障）、GET `/log`（**决策日志时间线**，`?limit=60`）、POST `/reset`

6.9 AI 文案 `AIController`（`/api/ai`）（5）

GET `/warning`、GET `/warning-bilingual`（中越双语）、POST `/predict-hazards`、DELETE `/predict-hazards`、GET `/route-risk`

6.10 其他业务

控制器（前缀）	端点	说明
`CustomsController`（`/api/customs`）4	GET `/efficiency`、POST/DELETE `/efficiency/{portId}`、DELETE `/efficiency`	口岸通关时效（baseHours 2.5h，normal/peak/holiday 1.0/1.4/1.8）
`KnowledgeController`（`/api/knowledge`）3	GET `（全量条目）`、`/search`、`/categories`	行业知识库检索（标题·关键词·正文**加权匹配，非向量检索**）
`BriefingController`（`/api/briefing`）2	GET+POST `/generate`	任务简报一键生成
`GeocodeController`（`/api/geocode`）4	GET `（?name=）`、`/list`、`/suggest`、`/snap`	本地地名库 → 天地图 POI → Nominatim 兜底；`/snap` 把坐标吸附最近路网节点。`/list` 同时下发 `nodeId`（该地名对应的路网内置节点）与 `hot`（常用地点权重），大屏起终点输入框据此在空输入时展开常用地点下拉，点选自带 `nodeId` 的条目不再发 `/snap`
`PlatformConfigController`（`/api/platform`）3	GET/PUT `/config`、POST `/template`	开放配置平台（千企千面：阈值档/货物/触达角色，改档后沙盘与方案同步重算）
`TiandituTileProxyController` 1	GET `/api/tianditu/{layer}/{z}/{x}/{y}.png`	天地图同源代理（密钥池轮转 + per-key 熔断 + 内存/磁盘两级缓存 + 30 天 immutable）
`PingController` 1	GET `/ping`	健康检查

> 异常统一由 `GlobalExceptionHandler` 转成带中文 `message` 的错误体（前端直接上屏，不靠状态码猜）。 护栏： `PublicDemoGuardFilter` **只拦 `/api/` 下的非 GET**（GET 与 SSE 全放行，节流 GET 等于节流演示），改全局状态的写接口 20 次/分/IP、其余写接口 180 次/分，超限 429。

七、数据文件与目录

路径	内容	是否入 gith
`../src/main/resources/data/vietnam-road-network.json`	清洗后的中越跨境路网拓扑（约 8.6 万节点 / 9.8 万边，含口岸虚拟边，~36 MB）	✅ 入库（保证 clone 即可演示）
`../src/main/resources/data/customs-efficiency.json`	口岸通关时效数据	✅
`../src/main/resources/data/knowledge-base.json`	行业知识库（RAG 检索源）	✅
`../src/main/resources/data/place-names.json`	本地地名库 52 条（与地图标注同源）：32 条带 `nodeId`（路网内置节点），14 条带 `hot` 权重（调度大屏输入框的“常用地点”下拉数据源）	✅
`decision-cache/*.json`	多智能体决策预生成缓存（`scenario_*.json`）+ 文案池 `recommendation-pool.json`	⛔ `../.gitignore`（随 `../asean-demo` 交付包分发）
`../data/tile-cache`	天地图瓦片磁盘缓存（img_w/cia_w/vec_w/cva_w）	⛔ 不入库
`../asean-demo/data/tile-cache`	离线交付包内的预热瓦片（实测 **29,493 张 / 247.4 MB**，img_w 12158 · cia_w 11043 · vec_w 3368 · cva_w 2924）	⛔ 不入库
`../data/sources`	原始地理数据源（OSM/自然地球/水系等）	⛔
`../tools/data`	加工作坊中间产物（矢量瓦片等）	⛔
`../frontend/dist`	前端构建产物，由后端静态托管	⛔
`.env.example`（根目录）	脱敏配置模板，不含任何凭证，供复现时复制为 `.env.local`	✅
`.env.local`（根目录）	真实密钥与 Token	⛔ **永不入库、不随压缩包外发**

八、模块结构（`../src/main/java/com/example/aseanweatherlogistics`）

- `controller/` **19 个 REST 控制器**（对应第六节 96 个端点）+ `GlobalExceptionHandler`（统一中文错误体）
- `config/` `WebConfig`（CORS 白名单 + dist 缓存头 + `/tiles/**` 双目录静态托管）、`PublicDemoGuardFilter`（公网演示护栏）
- `service/` **按职责分组（仅列主线）**：

- 算路与风险：`RouteService`（邻接表 + A* + 逐边惩罚硬熔断）、`RouteAgentService`（300 ms 防抖重算 + SSE 广播）、`GraphHopperRouteService` / `OsmDataLoader` / `CNRoadGeom`、`WeatherSimulator`（风险注入）、`RoadRiskIndex` / `TyphoonRiskRule` / `DriverRiskTipRule` / `WeatherTrendRule` / `CargoSafetyIndex`、`CorridorAdvisoryService`（全域受阻 DEFER/RETURN/HOLD）
- 多源气象：`WeatherSourceService`、`RealWeatherService`（节流+熔断）、`ContestWeatherProvider` / `ContestForecastService` / `OpenMeteoWeatherProvider` / `SimulatedWeatherProvider`、`WeatherWatchdogService`（`@Scheduled` 60 s 守护）
- 三智能体：**`DecisionOrchestrator`（真正的编排入口：串行调度 + 预生成缓存 + `agent-status` 广播 + 通道标签汇总）**、`RiskAgent` / `SolutionAgent` / `TouchAgent`、`RecommendationPool`（历史文案池轮换）
- 大模型通道：`OllamaService`（本地）/ `DeepseekService`（在线）/ `AIService`（预警文案与降级）、`AgentJson`（输出解析）
- 代价与方案：`CostCalculator`、`IntermodalService`、`DemoConfigService`（企业画像/阈值档）、`StartupWarmup`
- 水运：`WaterRiskEngine`（四条禁航红线）、`TributaryRiskService`、`waterway/`（`WaterwayDataService` + `Mock` / `Official` 双数据源）
- 业务支撑：`TaskService`（NONE/DISPATCHED/ACCEPTED/REJECTED）、`TripSignalService`（在途进度与新鲜度）、`DecisionSandboxService`（沙盘状态机与方向判定）、`DecisionLogService`（留痕）、`OutreachService`（分级叫应）、`CustomsEfficiencyService`、`KnowledgeBaseService`、`GeocodeService`、`BriefingService`
- **`repository/`** `RouteRepository` / `CustomsEfficiencyRepository` / `KnowledgeBaseRepository`（JSON 载入）
- **`model/`** `dto`（`RouteRequest` / `RouteResponse` / `RiskSegment` / `TransportTask`）、`entity`（`RoadNode` / `RoadEdge` / `KnowledgeEntry`）、`vo`（`WarningVO`）
- **`util/`** `GeoUtils`（球面距离）、`HttpUtil`、`DemoClock`（装船/到达等时刻基于真实时钟推导，不写死）

> 说明：仓库里曾有一个空的 `MultiAgentService.java`（无任何引用、也不参与编译产物），实际编排在 `DecisionOrchestrator`；旧文档里把它当编排层写是错的，本次一并清理。

九、服务设计可视化材料（图文对照）

方案的完整可视化见下列成品图（HTML 源与同名 PNG，高清图可直接用于展示）：

图号	文件	回答什么问题
图 1	`08-系统架构图.png`	气象从哪来 → 有什么用 → 怎么与 AI 交互 → 指导了什么（四段因果链）
图 2	`../../../桌面/织途者（材料提交）/服务设计可视化材料/09-系统架构图-分层版.png`	分层泳道与模块归属（PPT 主图）
图 3	`../../../桌面/织途者（材料提交）/服务设计可视化材料/10-服务蓝图.png`	五阶段 × 四泳道，粉线（互动线）/绿线（可视分离线）+ 分级叫应
图 4	`../../../桌面/织途者（材料提交）/服务设计可视化材料/11-界面原型图.png`	真实界面整屏 + 逐卡拆解 + 界面↔组件↔接口对照表
图 5	`../../../桌面/织途者（材料提交）/服务设计可视化材料/12-业务流程示意图.png`	主流程 12 步 + 四条分支 + 在途链路 + 断网降级链
图 6	`../../../桌面/织途者（材料提交）/服务设计可视化材料/13-数据使用说明图.png`	数据资产清单、流向、溯源与密钥边界

成图与截图的生产链（改完重跑即可，不依赖任何手工修图）：

```
node tools/_ui_shots.mjs        # CDP 驱动 headless Edge 打 8080 与 /driver.html，逐卡视口截图
node tools/_ui_pack.mjs         # 按交付名复制进 交付材料/_ui/
powershell -File tools/_arch_shot.ps1 -Pattern '11-*.html' -H 4150   # HTML → 同名 PNG（@2x）
node tools/_api_inventory.mjs  # 接口清单取证（第六节接口数据即由此产出）
```

十、依赖获取与网络访问矩阵

10.1 构建期依赖：获取渠道、体量与镜像

依赖	版本	获取渠道	国内直连	体量	加速 / 离线手段
Gradle	9.5.1	`../gradle/wrapper/gradle-wrapper.properties` 指向 services.gradle.org，`gradlew` 首次运行自动下载	可达，偶发慢	约 130 MB	把 distributionUrl 换为 `https://mirrors.cloud.tencent.com/gradle/gradle-9.5.1-bin.zip`，或预先把 zip 放进 `~/.gradle/wrapper/dists`
JDK	21	Gradle toolchain auto-detect；本机无匹配版本时 auto-download（Adoptium）	可达	约 200 MB	自装 JDK 21+；或在 `../gradle.properties` 指定 `org.gradle.java.home`
后端 Maven 依赖	见 3.1	Maven Central	可达	约 120 MB	在 `../build.gradle` 的 `repositories` 首位加 `maven { url 'https://maven.aliyun.com/repository/public' }`；处于公司/校园代理后还需配 `systemProp.http(s)ProxyHost/Port`
前端 npm 依赖	见 3.2	registry.npmjs.org；`../frontend/package-lock.json` 已入库	可达	约 200 MB（node_modules）	`npm config set registry https://registry.npmmirror.com`；精确复现用 `npm ci`
本地大模型	qwen2.5:14b	Ollama 安装包 + `ollama pull qwen2.5:14b`（registry.ollama.ai）	**通常需代理**	约 9 GB	从已拉取的机器拷贝 `~/.ollama/models`（manifest + blobs）；或跳过本地通道，推理自动落到文案池与模板
Android SDK / Gradle	Capacitor 8.5	developer.android.com 或 Android Studio	可达，建议走官方镜像	约 1 GB	**仅打 APK 需要**；浏览器打开 `/driver.html` 即是同一套司机端，不需要 Android 环境

预热两条命令（在能联网的机器上跑一次，缓存目录整体拷走即可离线复用）：

```
gradlew.bat dependencies --configuration runtimeClasspath   # 后端依赖落本地 Gradle 缓存
cd frontend && npm ci                                       # 前端依赖落 node_modules
```

10.2 运行期外部端点：谁要联网、谁要挂代理

端点	用途	凭证	网络要求	不可达时的系统行为
`180.149.156.4:8001`	大赛官方 CRA40 实况 / GOWFS 3h 预报	`WEATHER_REAL_TOKEN`（组委会下发）	需组委会侧网络可达，部分网络屏蔽该 IP:端口	连败 3 次熔断 300 s；自动降级 Open-Meteo，大屏「实时气象」标题显示真正服务的源
`api.open-meteo.com`	免凭证真实气象备源	无	国际公网，一般直连可达	再降级 `simulated` 离线模拟源，界面标「模拟降级」
`t0~t7.tianditu.gov.cn`	底图 1/2（影像 + 注记 / 矢量 + 注记）	`TIANDITU_KEYS`（**服务端类型**密钥 + 授权白名单）	中国境内直连	未配置密钥时 `/api/tianditu/**` 返回 503，前端按四级降级链切 OSM 在线 / 离线瓦片；已落盘瓦片不受影响
`nominatim.openstreetmap.org`	越南街道 / 店铺 POI 兜底	无（自定义 User-Agent）	**需出国际公网，通常要挂代理**：`GEOCODE_NOMINATIM_PROXY=127.0.0.1:<代理端口>`	仅影响越南街道联想；中国地名走天地图主通道，不受影响
`api.deepseek.com`	在线推理兜底	`DEEPSEEK_FALLBACK_KEY`	公网直连可达	Key 留空即视为通道未启用，请求不发本机，落到文案池 + 规则模板
`localhost:11434`	本地 Ollama	无	无需联网	连接超时 2 s，连败 2 次冷却 2 min 后转在线通道
手机 → `/driver.html`	司机端	无	与后端同一局域网即可，**不需要公网**	—

三条结论：

1. **主流程不需要任何凭证与外网**：路网拓扑、地名库、知识库、瓦片缓存、决策预生成缓存均随仓库或交付包分发，断网机器上算路、风险判定、禁航、分级叫应、司机端触达链路可完整跑通。
2. **需要联网才能核验的是“数据来源真实性”**：官方气象需 Token 与组委会侧网络，天地图需密钥与授权白名单，越南街道兜底需代理。三者不可达时界面显示降级后的真实来源，不做伪装。
3. **完全离线演示请直接用 `../asean-demo` 交付包**（内嵌 JRE + 2.9 万张预热瓦片 + 决策缓存；包内 `.env.local` 为便于开箱即可核验而内含申请人本人凭证，边界见 10.3），双击 `start.bat` 即可，不需要装 JDK / Gradle / Node。

10.3 脱敏对照与密钥边界

文档中的占位	真实环境变量 / 配置项	存放位置	获取途径	缺失能否复现主流程
`<气象TOKEN>`	`WEATHER_REAL_TOKEN` → `weather.real.token`	根目录 `.env.local`（已 gitignore）	组委会随参赛通知下发	能；降级 Open-Meteo 并如实标注
`<官方地址>`	`weather.real.base-url`	`application.properties` 内为真实地址（接口地址本身不是凭证）	参赛通知	能
`<TK1,TK2,...>`	`TIANDITU_KEYS` → `tianditu.keys`	根目录 `.env.local`	天地图开放平台申请**服务端类型**密钥，并把演示机出口 IP / 域名加入授权白名单	能；走包内预热瓦片与降级链
`<已授权域名>`	`tianditu.referer`	根目录 `.env.local`	与密钥授权范围一致	能
`<DEEPSEEK_KEY>`	`DEEPSEEK_FALLBACK_KEY` → `deepseek.fallback.key`	根目录 `.env.local`	DeepSeek 开放平台	能；本地 Ollama → 文案池 → 模板
`<本机代理端口>`	`GEOCODE_NOMINATIM_PROXY`	根目录 `.env.local`	本机代理的 HTTP 混合端口	能；仅少越南街道联想
`<待协议获取>`	`waterway.official.api-base` / `.token`	未申请，默认 `mock`	广西水运调度平台无自助 API，需合作协议	能；`mock` 源手动推送验证同一条链路
`192.168.200.140`	`../frontend/capacitor.config.json` 的 `server.url`	已入库（局域网地址，不是凭证）	开发机当时的内网 IP	能；换机器时改成当前后端 IP，或直接用浏览器打开 `/driver.html`

密钥边界声明：

- 仓库内**被版本控制的文件不含任何真实凭证**：`application.properties` 中所有凭证项一律写成 `${VAR:}` 空默认值；`../asean-demo/`（离线演示包目录）整体被 `.gitignore` 排除，不入库。复核命令（无输出即无凭证）：

``bash git grep -n -E "(TOKEN|_KEY|KEYS|_TK)[=:][0-9a-zA-Z]{8}" git grep -n -E "sk-[0-9a-f]{16,}"``

- 真值只存在于开发机根目录 `.env.local`，该文件被 `../.gitignore` 排除；**提交或压缩外发整个文件夹时请勿包含它。**
- 可运行 Demo 包（`../asean-demo/.env.local`，exe 与 zip 同源）为便于开箱即可核验官方气象、影像底图与真实模型推理，**内含申请人本人的三份凭证**（组委会气象 Token、天地图密钥池、DeepSeek Key）。该目录不入库，本包只定向交付，不得上传公开网盘或群发链接。
- 早期版本的外发离线包曾以明文携带过同类凭证，且当前版本仍选择随包外发，因此这些凭证一律应视为已扩散：**展示结束后即在天地图 / DeepSeek 控制台作废并重新申请**，新值只写入本机 `.env.local`，不写进文档与仓库。源代码与技术文档本身仍保持脱敏处理（上述占位表即为脱敏版）。

十一、从零复现与常见故障

11.1 从零复现（不填任何密钥）

```
# 0) 环境自检
java -version            # 需要 21+；没有也行，Gradle toolchain 会自动下载
git --version
node -v                  # 需要 20+

# 1) 取源码
git clone https://github.com/zhige12/asean-weather-logistics.git
cd asean-weather-logistics
copy .env.example .env.local        # 全部留空即可，官方数据要核验时再填自己的凭证

# 2) 后端编译 + 单元测试
gradlew.bat compileJava compileTestJava test

# 3) 前端构建（后端从 file:frontend/dist 托管页面）
cd frontend && npm ci && npm run build && cd ..

# 4) 启动
gradlew.bat bootRun
```

预期结果与自检点：

检查项	命令 / 操作	预期
服务存活	`curl http://localhost:8080/ping`	`OK`
路网就绪	启动日志	`StartupWarmup` 加载 8.6 万节点 / 9.8 万边完成，首屏 10–30 s
大屏	浏览器打开 `http://localhost:8080/`	地图出图、路线可规划、风险卡有数据
地名下拉	点一下起点输入框不输字	展开 14 条常用地点（来自地名库 `hot` 标记，零网络配额）；点选即填入节点 id
司机端	打开 `http://localhost:8080/driver.html`	接单 / 在途上报 / 触达弹窗可用
实时通道	大屏派单后看司机端	SSE `/api/agent/events` 推 `task-assigned`，无需刷新
气象源	`curl http://localhost:8080/api/weather/source`	未填 Token 时 `selectedSource=contest-observation`、`servedSource=open-meteo`（两者分开回显，不假装官方）
节流审计	`curl http://localhost:8080/api/weather/rate-stats`	四个计数（真实回源 / 缓存命中 / 节流拦下 / 熔断打开）
天地图代理	浏览器请求 `/api/tianditu/img_w/7/103/74.png`	未配密钥返回 503，前端自动降级底图 3/4
推理通道	大屏三智能体汇总标签	无 Ollama、无 Key 时显示「文案池 / 模板」

11.2 环境差异与已知坑

现象	原因	处理
编译报找不到符号 `CorridorAdvisoryService` / `RecommendationPool`	新增源文件未随提交入库	确认 `src/main/java/.../service/` 下这两个类与 `../frontend/src/utils/corridorAdvisory.js`、`src/test/.../SolutionAgentDetourTriggerTest.java` 均在提交物内
`Unsupported class file major version` / toolchain 报错	本机 JDK 低于 21	装 JDK 21+，或让 toolchain 自动下载（`../gradle.properties` 已开 auto-download）
页面改了但没变化	Spring 托管的是已构建的 `../frontend/dist`	重新 `npm run build`，浏览器 Ctrl+F5
8080 被占用	端口冲突	`gradlew.bat bootRun --args='--server.port=8090'`；离线包 `start.bat` 会自动在 8080–8089 找空端口
底图全空	既无密钥也无预热瓦片	属预期降级；需离线出图请填 `TIANDITU_KEYS` 或直接使用 `../asean-demo` 包
越南街道搜不到	Nominatim 需出国际公网	填 `GEOCODE_NOMINATIM_PROXY`，或置 `GEOCODE_NOMINATIM_ENABLED=false`
写接口返回 429	公网演示护栏限流	属设计行为；单人演示机可上调 `DEMO_GUARD_STRICT_LIMIT` / `NORMAL_LIMIT`
点击普遍变慢	内存热缓存过大导致小内存机器频繁换页	调小 `tianditu.mem-cache-entries`（离线包已设 5000）
Android 端白屏	`server.url` 指向的电脑 IP 变了，或手机与电脑不在同一 WiFi	改 `../frontend/capacitor.config.json` 后 `npx cap sync android` 重装；或直接用手机浏览器打开 `/driver.html`